UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA

REPORTE DE PRÁCTICA Nº 06

NOMBRE COMPLETO: Rodríguez Zacarias Ivan

Nº de Cuenta: 320340020

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 6

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 26 de Marzo de 2026

CALIFICACIÓN: _____

Ejercicio

Utilizando GIMP para edición de imágenes se crearon texturas para manejarlas tanto dentro del código de OpenGL para texturizas primitivas (cubos y octaedros), y también para utilizarlas directamente con modelos importados desde archivos fbx y obj exportados desde blender. Los modelos creados para el ejercicio de la práctica son un octaedro texturizado a partir de vértices y código y el coche con texturas en el visor y el cofre.

```
void CrearOctaedro()
{
    float a = 0.7071f;
    float n = 0.5773f;

    unsigned int octa_indices[] = {
        0, 1, 2,
        3, 4, 5,
        6, 7, 8,
        9, 10, 11,
        12, 13, 14,
        15, 16, 17,
        18, 19, 20,
        21, 22, 23
    };

    GLfloat octa_vertices[] = {
        // x      y      z        S      T        NX     NY     NZ
        // 1
        0.0f,  a,     0.0f,    0.32f,  0.2f,     n,     n,     n,
        a,     0.0f,  0.0f,    0.01f,  0.01f,    n,     n,     n,
        0.0f,  0.0f,  a,       0.01f,  0.39f,    n,     n,     n,

        // 4
        0.0f,  a,     0.0f,    0.34f,  0.2f,     n,     n,    -n,
        0.0f,  0.0f, -a,       0.65f,  0.385f,   n,     n,    -n,
        a,     0.0f,  0.0f,    0.65f,  0.015f,   n,     n,    -n,

        // 3
        0.0f,  a,     0.0f,    0.34f,  0.205f,  -n,     n,    -n,
       -a,     0.0f,  0.0f,    0.34f,  0.59f,   -n,     n,    -n,
        0.0f,  0.0f, -a,       0.65f,  0.395f,  -n,     n,    -n,

        // 2
        0.0f,  a,     0.0f,    0.33f,  0.205f,  -n,     n,     n,
        0.0f,  0.0f,  a,       0.02f,  0.41f,   -n,     n,     n,
       -a,     0.0f,  0.0f,    0.32f,  0.59f,   -n,     n,     n,

       // 8
       0.0f, -a,     0.0f,    0.67f,  0.8f,     n,    -n,     n,
       0.0f,  0.0f,  a,       0.99f,  0.99f,    n,    -n,     n,
       a,     0.0f,  0.0f,    0.99f,  0.61f,    n,    -n,     n,
```

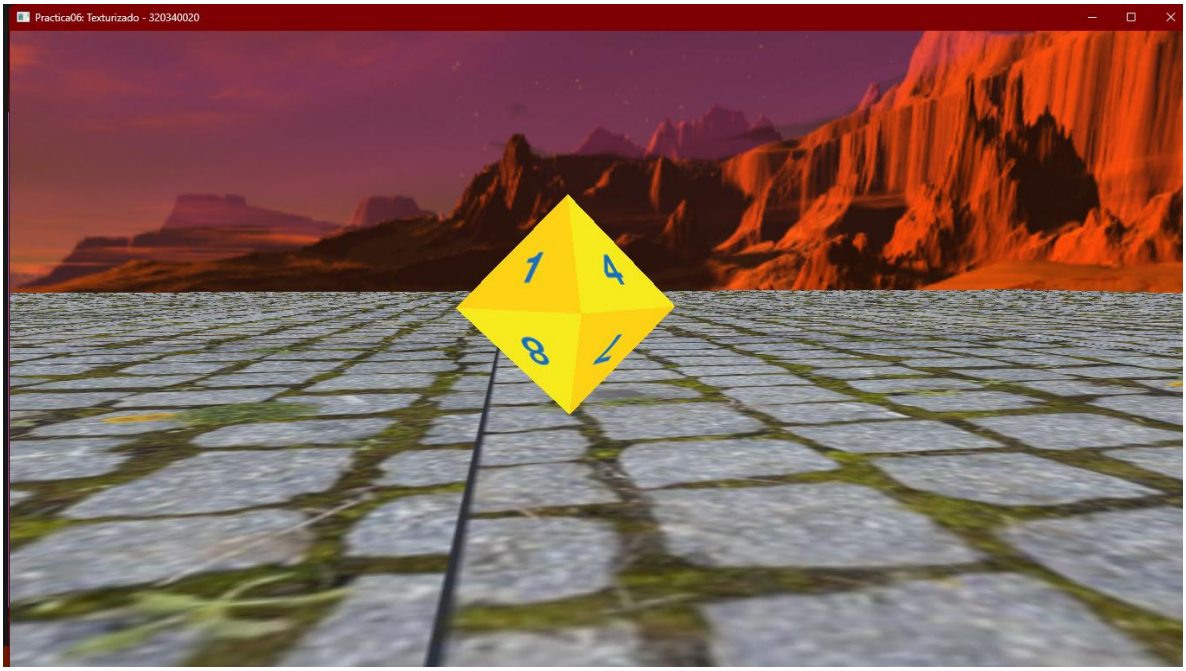
```
        // 7
        0.0f, -a,     0.0f,    0.67f,  0.79f,    n,    -n,    -n,
        a,     0.0f,  0.0f,    0.98f,  0.59f,    n,    -n,    -n,
        0.0f,  0.0f, -a,       0.67f,  0.43f,    n,    -n,    -n,

        // 6
        0.0f, -a,     0.0f,    0.645f,  0.785f,   -n,    -n,    -n,
        0.0f,  0.0f, -a,       0.65f,  0.41f,   -n,    -n,    -n,
       -a,     0.0f,  0.0f,    0.34f,  0.6f,    -n,    -n,    -n,

       // 5
       0.0f, -a,     0.0f,    0.63f,  0.785f,   -n,    -n,     n,
      -a,     0.0f,  0.0f,    0.34f,  0.61f,   -n,    -n,     n,
       0.0f,  0.0f,  a,       0.34f,  0.99f,   -n,    -n,     n,
    };

    Mesh* octaedro = new Mesh();
    // 192 floats, 24 indices
    octaedro->CreateMesh(octa_vertices, octa_indices, 192, 24);
    meshList.push_back(octaedro);
```

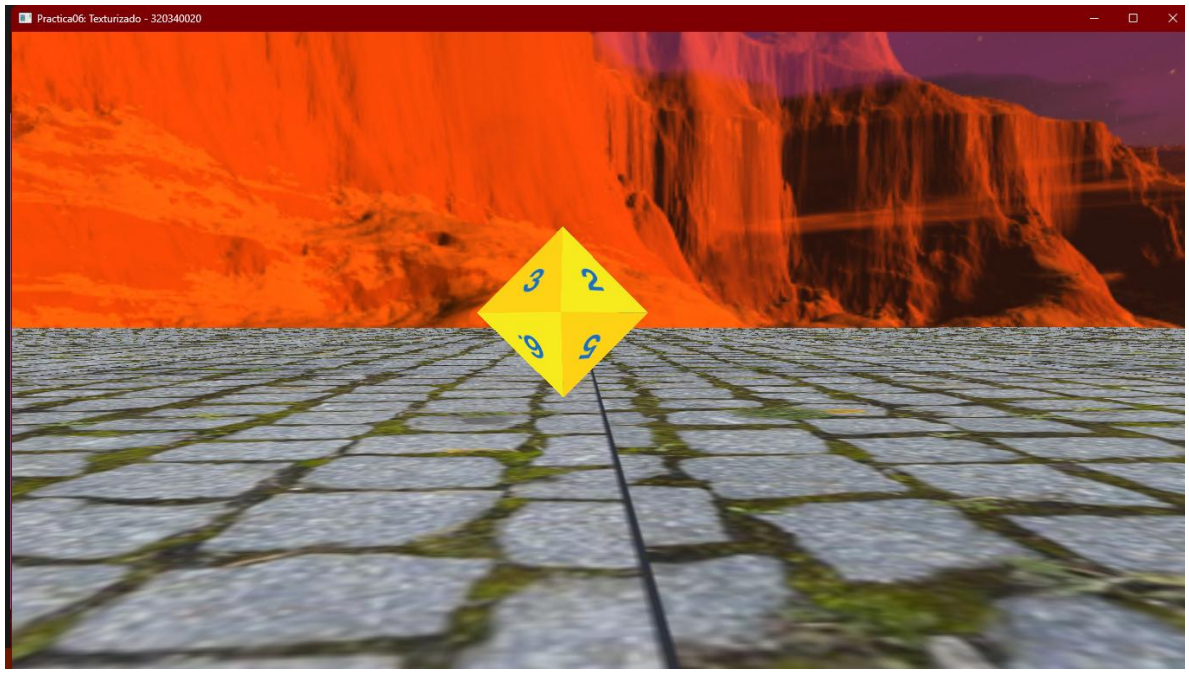
```
//Octaedro
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 4.5f, -2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
octaTexture.UseTexture();
meshList[5]->RenderMesh();
```

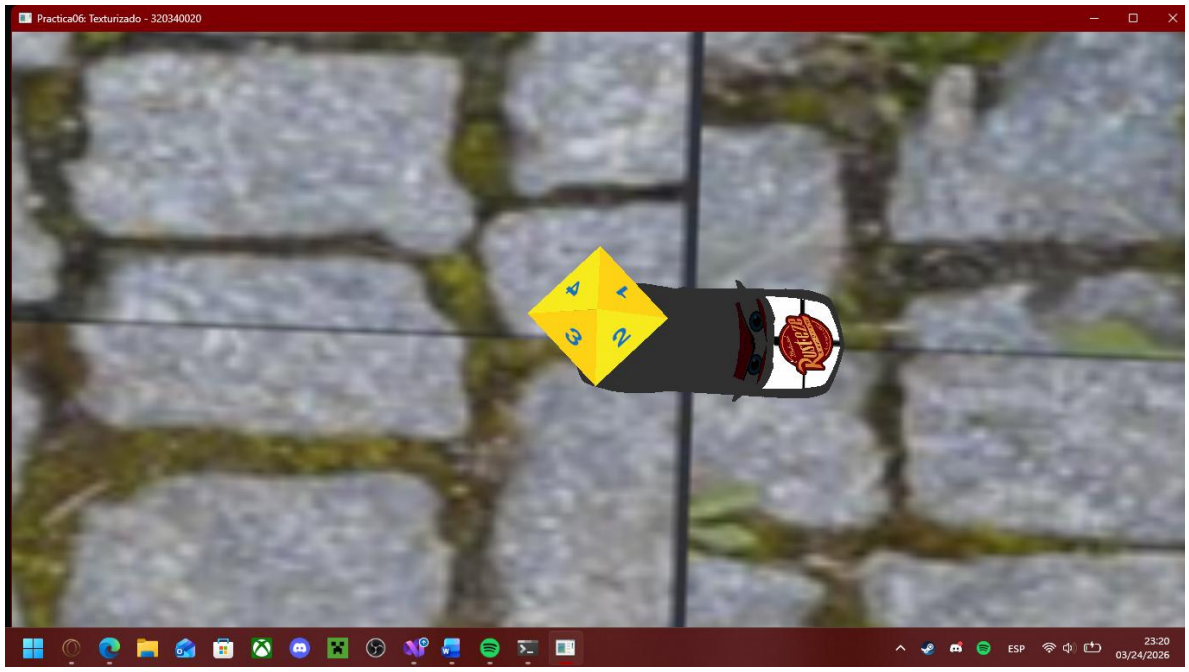
```
///Camaro
model = glm::mat4(1.0);
color = glm::vec3(0.19f, 0.19f, 0.19f); //Cuerpo del Auto
model = glm::translate(model, glm::vec3(0.0f, 0.2f, 0.0f));
modelaux = model;
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Camaro.RenderModel();

//Ruedas
model = modelaux;
color = glm::vec3(1.0f, 1.0f, 1.0f);
model = glm::translate(model, glm::vec3(0.79f, -0.45f, 1.45f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Llanta_M.RenderModel();

model = modelaux;
color = glm::vec3(1.0f, 1.0f, 1.0f);
model = glm::translate(model, glm::vec3(-0.79f, -0.45f, 1.45f));
model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 1.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Llanta_M.RenderModel();

model = modelaux;
color = glm::vec3(1.0f, 1.0f, 1.0f);
model = glm::translate(model, glm::vec3(0.79f, -0.45f, -1.45f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Llanta_M.RenderModel();
```

```
model = modelaux;
color = glm::vec3(1.0f, 1.0f, 1.0f);
model = glm::translate(model, glm::vec3(-0.79f, -0.45f, -1.45f));
model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 1.0f, 0.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Llanta_M.RenderModel();

//Cofre
model = modelaux;
color = glm::vec3(1.0f, 1.0f, 1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.22f, 1.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
CamaroC.RenderModel();
```

Resultado final:




Practica06: Texturizado - 320340020


Practica06: Texturizado - 320340020
ESP
23:20
03/24/2026

Practica06: Texturizado - 320340020

Practica06: Texturizado - 320340020
Rust-eze
Bumper Ointment
Rear End Formula

Conclusión.

Se aprendieron 2 métodos de texturizado, evidentemente el método utilizado mediante código es el precursor a la manera de importarlos mediante objetos obj o fbx los cuales son compatibles con las librerías que se utilizaron de OpenGL.

Por ahora creo que el tipo modelo que más me sirve a la hora de exportarlo desde blender es el obj, tendré que explorar más la salida de los fbx para obtener resultados similares al obj.

Bibliografía y Referencias

Ing. José Roque RG (2026). Laboratorio de Computación Gráfica e Interacción Humano Computadora. Clase. Facultad de Ingeniería UNAM.